
第 7 章

Statsmodels 及其基本使用

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

7. Statsmodels 及其基本使用（新版讲义）

本页是第 7 章的章首页（索引）。正文位于 01-方差分析与回归.md ~ 04-常见误区与技巧.md，与原版资料分离（原版见../原始资料/chap7/）。

本章概览

Statsmodels 是 Python 中一个面向**统计建模与计量经济学**的库：它在 Pandas/NumPy 之上提供了回归、方差分析、广义线性模型、时间序列分解与 ARIMA 预测等经典统计工具，并给出完整、规范的**统计推断结果**（系数标准误、t 值、P 值、R²、F 检验、假设检验统计量等）。与主要面向**预测/机器学习**的 scikit-learn 不同，Statsmodels 更强调**模型的可解释性、诊断与显著性检验**，非常适合教学、论文与业务归因分析。

本章内容：

- 用 OLS / ols 做**线性回归**，并把公式 API 与数组 API 对比；
- 用 anova_lm 做**单因素方差分析**，理解它与回归的统一框架；
- 用 GLM 做**广义线性模型**（以二分类 Logit 为例）；
- 用 seasonal_decompose 做**时间序列分解**；
- 用 adfuller 做**平稳性检验**；
- 用 ARIMA 做**时间序列预测**。

学习目标

学完本章，你应该能够：

1. 用 statsmodels.api.OLS 完成一元/多元最小二乘回归，读懂 summary() 中的系数、标准误、t 值、P 值、R²、F 检验。
2. 使用公式 API ols('y ~ x1 + x2' , data=df) 进行回归，并理解 C(...)、I(...) 等公式写法。
3. 用 anova_lm 做单因素方差分析，解读 F 统计量与 P 值，说明 ANOVA 与回归的统一性。
4. 用 GLM + Binomial/Logit 拟合二分类数据，并用 predict 得到概率。
5. 用 seasonal_decompose 分解时间序列为趋势、季节、残差；用 adfuller 判断平稳性。
6. 用 ARIMA 拟合并预测未来值，能读懂 SARIMAX Results 摘要。

案例索引

案例卡	所在节	技术点	难度
1. 单因素方差分析——三种肥料对产量差异是否显著	01 方差分析与回归	C()、anova_lm、F/P 值	★★☆
2. OLS 多元回归——用公式接口解读系数与 p 值	01 方差分析与回归	ols 公式、pvalues、conf_int	★★☆
3. ARIMA 时间序列——用 ADF 检验与残差诊断选 p,d,q	02 时间序列预测	adfuller、ACF/PACF、AIC、残差诊断	★★★
综合案例：城市日负荷的统计建模与预测	03 综合案例	ANOVA + OLS + 分解/ARIMA + RMSE 对比	★★★

先修要求与运行环境

- 熟悉 Python 基础语法、NumPy 数组与 Pandas DataFrame（前几章内容）。

- 安装 Statsmodels（与安装 NumPy/Pandas 一起）:

```
pip install statsmodels
# 国内镜像（更快）
pip install statsmodels -i https://pypi.tuna.tsinghua.edu.cn/simple
```

- 本章还需要 matplotlib（绘图）与 scipy（统计），若未安装请一并安装:

```
pip install matplotlib scipy
```

- 推荐在 JupyterLab / VS Code 中打开.ipynb 练习；命令行列出的脚本保存为.py 运行即可。

本仓库已在 statsmodels 0.14.6、NumPy 2.1.2、Pandas 2.2.3 环境下验证全部示例。

本章目录

小节	文件	内容
01 方差分析与回归	01-方差分析与回归.md	OLS 回归、公式 API、anova_lm、GLM、诊断、WLS；案例卡 1/2
02 时间序列预测	02-时间序列预测.md	seasonal_decompose、ADF、ARIMA；案例卡 3
03 综合案例	03-综合案例.md	城市日负荷：方差分析 + 回归 + 时间序列预测
04 常见误区与技巧	04-常见误区与技巧.md	易错点表格、性能/调试/自测清单

数学预备与附录

本章的统计背景已集中到附录《数学与算法补充》:

- **C 概率统计基础**: C.3 统计推断（检验/置信区间）、C.4 回归与方差分析、C.5 时间序列；
- [打开附录 C](#) | [附录索引](#)

讲回归/ANOVA/时序前，用 C.4/C.5 讲 5 分钟“模型假设”；期末大作业前回看。

练习与上机入口

- [本章练习 \(exercises/\)](#)：10 道自测题、作业、参考答案。
- [本章上机 \(lab/\)](#)：19 个代码单元、4 个大 Part，含案例卡跟练/变形/综合任务与综合任务。
- [本章参考与延伸阅读 \(references.md\)](#)：官方文档、精品教程、习题集、中文资料。
- [教学说明 \(teaching.md\)](#)：课时安排、重点难点、考核建议（教师用）。

建议课时

环节	学时	对应内容
讲课	3-4 学时	01-02 正文 + 03 案例讲解
上机	2-3 学时	lab/ 逐题完成；课后完成 exercises/ 作业

使用说明

- **学生**：先读 01–02 正文并运行代码 → 完成 lab/ 上机 → 提交 exercises/ 作业 → 用 quiz 自测。统计概念/公式不清楚时，到 references.md 找官方对应章节。
- **教师**：按 teaching.md 的课时表讲；lab 可作为上机课内容；quiz.ipynb 可回收批改；03-综合案例.md 可作课堂讨论。

01 方差分析与回归

本节对应原版 7.1 的内容，并增补目标、诊断、GLM、WLS 与思考题。配套图：images/regression_ols.png、images/anova_boxes.png、images/qq_resid.png（由 build/make_chap7_figures.py 生成）。

本节目标

- 会用 statsmodels.api.OLS 与 add_constant 做一元/多元线性回归，并读懂 summary();
- 会用公式 API ols('y ~ x' , data=df), 理解分类变量 C(...) 与交互项写法;
- 会用 anova_lm 做单因素方差分析，解读 F 统计量与 P 值;
- 会用 GLM + Binomial 做二分类 Logit 回归并得到概率;
- 会做基本回归诊断（残差 QQ 图、JB 统计量）与异方差处理（WLS）。

先修

- Python 基础、NumPy 数组、Pandas DataFrame; 了解最小二乘与回归的统计含义。
- 已安装 statsmodels（见章 README）。

官方文档/参考入口

内容	链接	说明
OLS（回归模型）	链接	最小二乘回归主接口
Regressions 官方教程	链接	回归模型总览
公式 API	链接	ols('y ~ x' , data=...) 语法
anova_lm	链接	对 OLS 结果做方差分析
GLM	链接	广义线性模型
WLS	链接	加权最小二乘

1 方差分析（ANOVA）

1.1 概念

方差分析（ANOVA, Analysis of Variance）用来检验**多个组的均值是否显著不同**。其核心思路是把总离差平方和 SS_{tot} 分解为**组间平方和 $SS_{between}$** 与**组内平方和 SS_{within}** ，并构造 F 统计量：

$$F = \frac{SS_{between}/(k - 1)}{SS_{within}/(N - k)}$$

其中 k 为组数， N 为总样本数。若 F 很大（对应 P 值很小），说明组间差异显著，各组的总体均值不全相等。Statsmodels 通常先用 OLS 拟合一个“将分组变量视为分类自变量”的模型，再用 anova_lm 输出方差分析表。

1.2 单因素方差分析示例

构造三个组（A/B/C），每组均值分别为 0、1、2，并加噪声：

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

rng = np.random.default_rng(1)
n = 12
df = pd.DataFrame({
    'group': ['A']*n + ['B']*n + ['C']*n,
    'y': np.r_[rng.normal(0,1,n), rng.normal(1,1,n), rng.normal(2,1,n)]
})
print(df.groupby('group')['y'].mean().round(4))

model = ols('y ~ C(group)', data=df).fit()
anova_res = sm.stats.anova_lm(model, typ=1)
print(anova_res)
```

运行结果：

```
group
A    0.2354
B    0.9966
C    1.9345
Name: y, dtype: float64
```

	df	sum_sq	mean_sq	F	PR(>F)
C(group)	2.0	17.386102	8.693051	9.297656	0.000627
Residual	33.0	30.854088	0.934972	NaN	NaN

1.3 解读

- C(group) 告诉 ols 把 group 当作**分类变量**，展开成虚拟变量；
- PR(>F) 是 F 检验的 P 值，本例 $0.000627 < 0.05$ ，说明三组均值**存在显著差异**；
- sum_sq 是平方和，mean_sq = sum_sq / df，F = 组间 mean_sq / 组内 mean_sq。

多做几组对比时，anova_lm 的 typ 可切换 I/II/III 型平方和；单因素情形三者等价。

案例卡 1：单因素方差分析——三种肥料对产量差异是否显著

目标

在田间试验里，把同一作物种在 45 块小地上，分别施 A、B、C 三种肥料，每种肥料 15 块地，记录每块地产量。我们要用**单因素方差分析**（One-Way ANOVA）回答：三种肥料的平均产量差异，究竟是**真实的处理效应**，还是**抽样噪声**造成的巧合。核心统计量是 F 值与其 P 值。

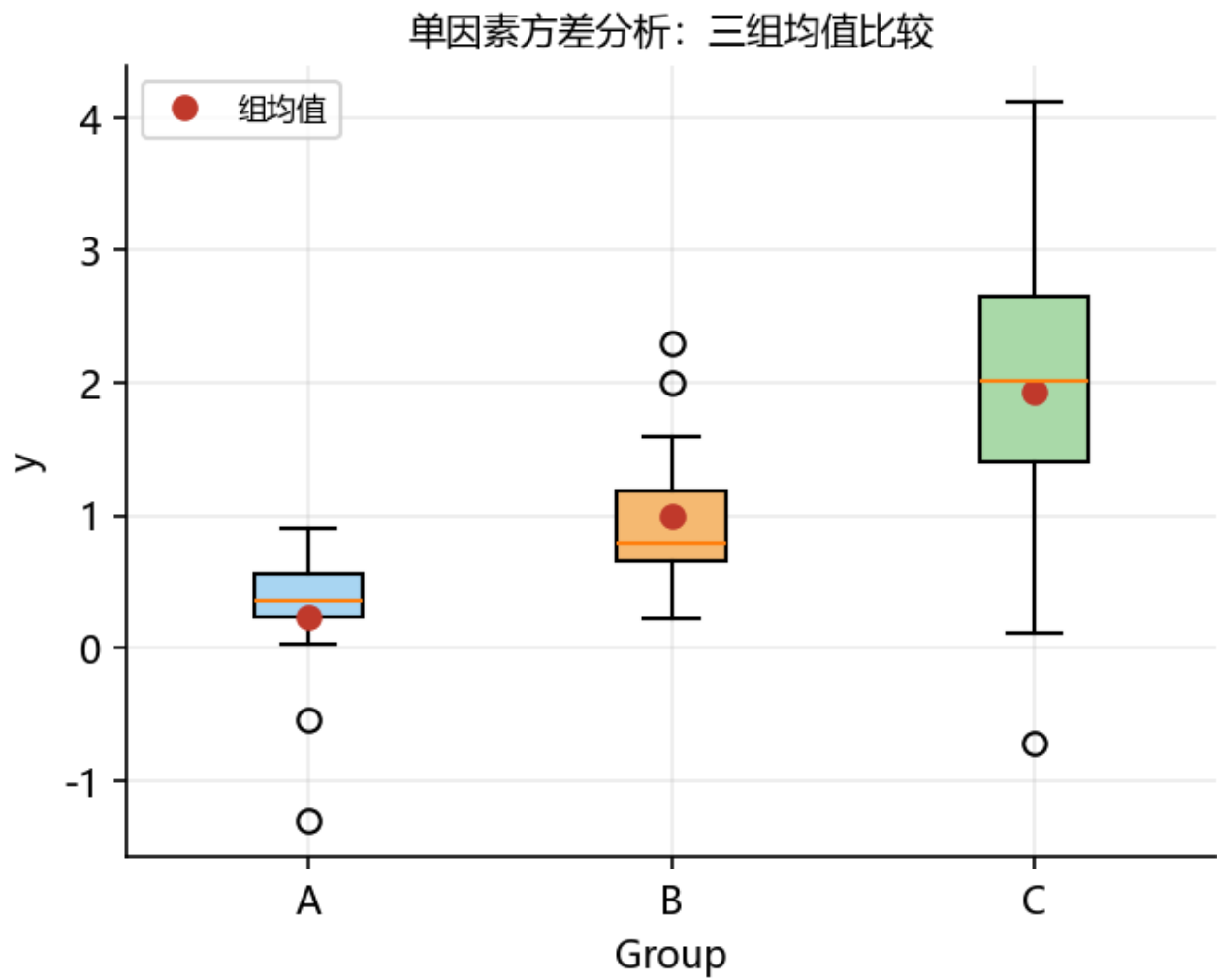


Figure 1: 三个组的箱线图

代码

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
import matplotlib.pyplot as plt
import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)
pd.set_option('display.width', 100)

rng = np.random.default_rng(42)
n = 15
a = rng.normal(50, 6, n)
b = rng.normal(55, 6, n)
c = rng.normal(60, 6, n)
df = pd.DataFrame({'fertilizer': ['A']*n + ['B']*n + ['C']*n,
                    'out': np.r_[a, b, c]})

print(df.groupby('fertilizer')['out'].mean().round(2))

model = ols('out ~ C(fertilizer)', data=df).fit()
anova = sm.stats.anova_lm(model, typ=1)
print('F = %.3f' % anova['F'].iloc[0])
print('P = %.3g' % anova['PR(>F)'].iloc[0])
print('SS_between = %.3f' % anova['sum_sq'].iloc[0])
print('SS_within = %.3f' % anova['sum_sq'].iloc[1])
print('df_between =', anova['df'].iloc[0], ' df_within =', anova['df'].iloc[1])

os.makedirs('images', exist_ok=True)
plt.figure(figsize=(5.6, 3.8))
bp = plt.boxplot([a, b, c], tick_labels=['A', 'B', 'C'], patch_artist=True)
for patch, color in zip(bp['boxes'], ['#a8d5f2', '#f5b971', '#a9d9a8']):
    patch.set_facecolor(color)
plt.plot([1, 2, 3], [a.mean(), b.mean(), c.mean()], marker='o', color='#c0392b',
         ls='', label='组均值')
plt.legend(fontsize=8)
plt.grid(alpha=0.25)
plt.savefig('images/case1_fertilizer_anova.png', dpi=150)
plt.close()
print('saved images/case1_fertilizer_anova.png')
```

运行结果 (已运行核验)

```
fertilizer
A    49.98
B    55.22
C    60.80
Name: out, dtype: float64
F = 18.455
P = 1.77e-06
```

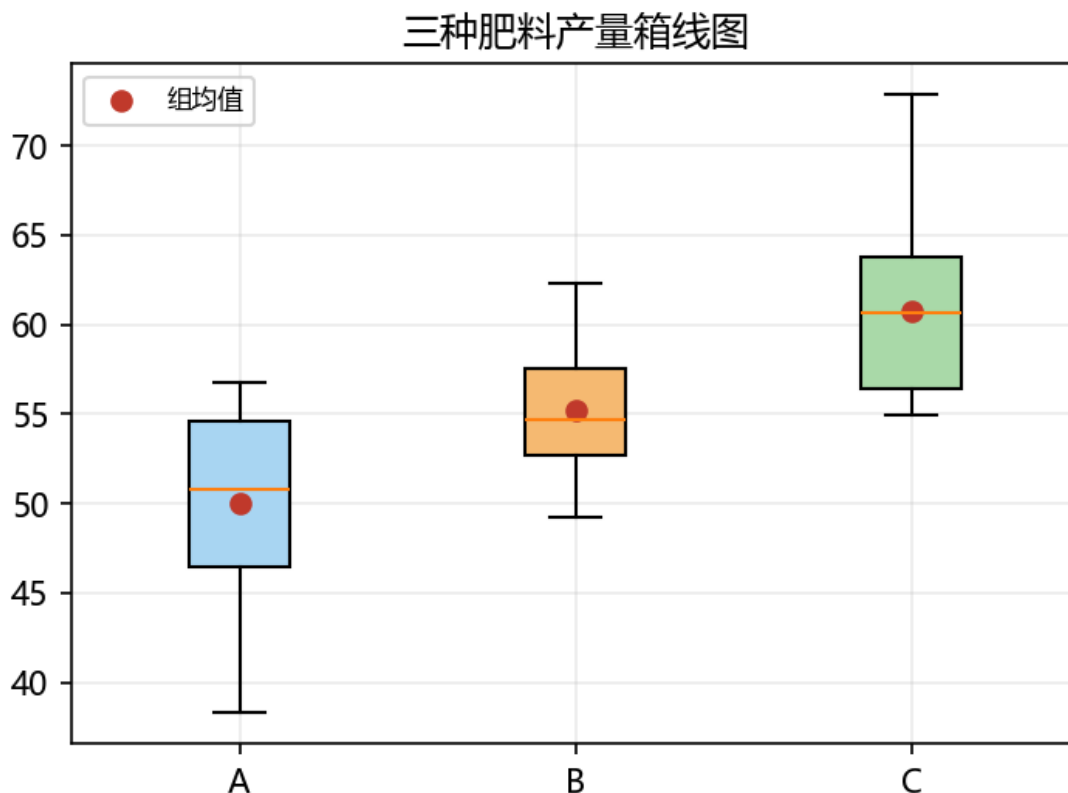


Figure 2: 三种肥料的产量箱线图

```
SS_between = 877.288
SS_within = 998.289
df_between = 2.0 df_within = 42.0
saved_images/case1_fertilizer_anova.png
```

讲解

一句话解释：把 45 块地的产量按“肥料”分成三堆，先看每堆平均数，再看堆与堆之间的差距，是否大到不像是“随机抽签运气好/坏”造成的。

1. `rng.normal(50, 6, n)`: 生成 15 个“平均 50、标准差 6”的产量，模拟 A 肥；B、C 分别把均值换成 55、60。固定 `seed=42` 保证每次结果一致。
2. `df.groupby('fertilizer')['out'].mean()`: 把产量按肥料分组求平均，得到 $A \approx 49.98$ 、 $B \approx 55.22$ 、 $C \approx 60.80$ ，肉眼能看出递增。
3. `ols('out ~ C(fertilizer)', data=df)`: 把“肥料”当分类自变量 (`C(...)` 自动转成哑变量)，拟合线性模型；这就是 ANOVA 的回归写法。
4. `sm.stats.anova_lm(model, typ=1)`: 对该回归结果做方差分析，把总平方和拆成“组间”与“组内”两块，算出 F 与 P 值。
5. SS_{between} (组间) ≈ 877.29 、 SS_{within} (组内) ≈ 998.29 ，说明三组之间的差异占总差异的一半左右，不是噪声。
6. $F=18.455$ 、 $P=1.77e-06 < 0.05$: 拒绝“三组均值相同”的原假设，即三种肥料对产量的影响**统计显著**。

主要用法 / API

需求	写法	说明
生成正态随机数	<code>rng.normal(mean, std, n)</code>	模拟每组产量
按组求均值	<code>df.groupby('fertilizer')['out'].mean()</code>	组间比较第一步
分类变量进公式	<code>ols('out ~ C(fertilizer)', data=df)</code>	C() 把类别变哑变量
方差分析表	<code>sm.stats.anova_lm(model, typ=1)</code>	输出 df/平方和/F/P
取 F / P 值	<code>anova['F'].iloc[0] / anova['PR(>F)'].iloc[0]</code>	用 .iloc[0] 避免位置弃用警告
画箱线图	<code>plt.boxplot([a, b, c], tick_labels=...)</code>	直观展示组分布与均值

常见错误

- 1. 误把“组均值看起来不同”当成“显著不同”：还要看组内波动 (SS_within) 够不够小；组内波动大时，均值差异可能不显著。
- 2. 公式里直接写 `ols('out ~ fertilizer', ...)` 而不加 `C()`：字符串列会被 patsy 当连续变量或报错。
- 3. 直接用 `anova['F'][0]` 取第一行：新版 pandas 可能给出“按位置取数”的 FutureWarning，应改用 `.iloc[0]`。
- 4. 忘记在绘图前 `os.makedirs('images', exist_ok=True)`：首次没有 images/ 目录时 savefig 会报错。

拓展 / 跟练

- 1. 把 n 改成 30，观察 F 与 P 值如何变化；把种子改成别的值，多次运行看结论是否稳定。
- 2. 用 `scipy.stats.f_oneway(a, b, c)` 直接计算，和 `anova_lm` 的 F/P 对比是否一致。
- 3. 给数据再加一个“对照”组（不施肥、均值 45），看 4 组 ANOVA 结论。
- 4. 用 `statsmodels.stats.multicomp.pairwise_tukeyhsd` 做 Tukey 事后两两比较，找出哪两组差异显著。

2 线性回归 (OLS)

2.1 数组接口：sm.OLS

数组接口要求手动加载截距列 (add_constant)，适合已用 NumPy 准备数据的场景。

```
import numpy as np
import statsmodels.api as sm

rng = np.random.default_rng(0)
x = np.linspace(0, 10, 50)
y = 2*x + 1 + rng.normal(0, 1, size=len(x))

X = sm.add_constant(x.reshape(-1, 1))
model = sm.OLS(y, X).fit()
```

```
print("params:", model.params)
print("R²:", model.rsquared)
print(model.summary())
```

运行结果 (Date/Time 随运行时间变化):

```
params: [0.54884546 2.1160302 ]
R²: 0.9819621414581136
```

```

                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.982
Model:                  OLS    Adj. R-squared:           0.982
Method:                 Least Squares    F-statistic:       2613.
Date:                   Wed, 26 Aug 2026    Prob (F-statistic): 1.63e-43
Time:                   14:08:23    Log-Likelihood:    -62.504
No. Observations:       50    AIC:                  129.0
Df Residuals:           48    BIC:                  132.8
Df Model:                1
Covariance Type:        nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
const                0.5488      0.240         2.285      0.027      0.066      1.032
x1                   2.1160      0.041        51.118      0.000      2.033      2.199
=====
Omnibus:              0.794    Durbin-Watson:       1.565
Prob(Omnibus):         0.672    Jarque-Bera (JB):     0.889
Skew:                  -0.229    Prob(JB):             0.641
Kurtosis:              2.533    Cond. No.             11.7
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

解读要点:

- 截距 const ≈ 0.549 , 斜率 x1 ≈ 2.116 , 接近真实值 (1, 2);
- $R^2=0.982$: 模型解释了约 98.2% 的方差; Adj. R^2 对自变量个数做了惩罚;
- F-statistic 与 Prob(F) 是整体显著性检验; Prob(F) $\approx 1.6e-43$, 模型整体显著;
- 每个系数的 P>|t| 是该变量是否显著 (0.05 为常用阈值);
- [0.025, 0.975] 是 95% 置信区间。

2.2 公式接口: ols

把数据放进 DataFrame, 即可用公式写模型:

```
import pandas as pd
from statsmodels.formula.api import ols

x = np.linspace(0, 10, 50)
y = 2*x + 1 + rng.normal(0, 1, size=len(x))
df2 = pd.DataFrame({'x': x, 'y': y})
fm = ols('y ~ x', data=df2).fit()
print(fm.params)
```

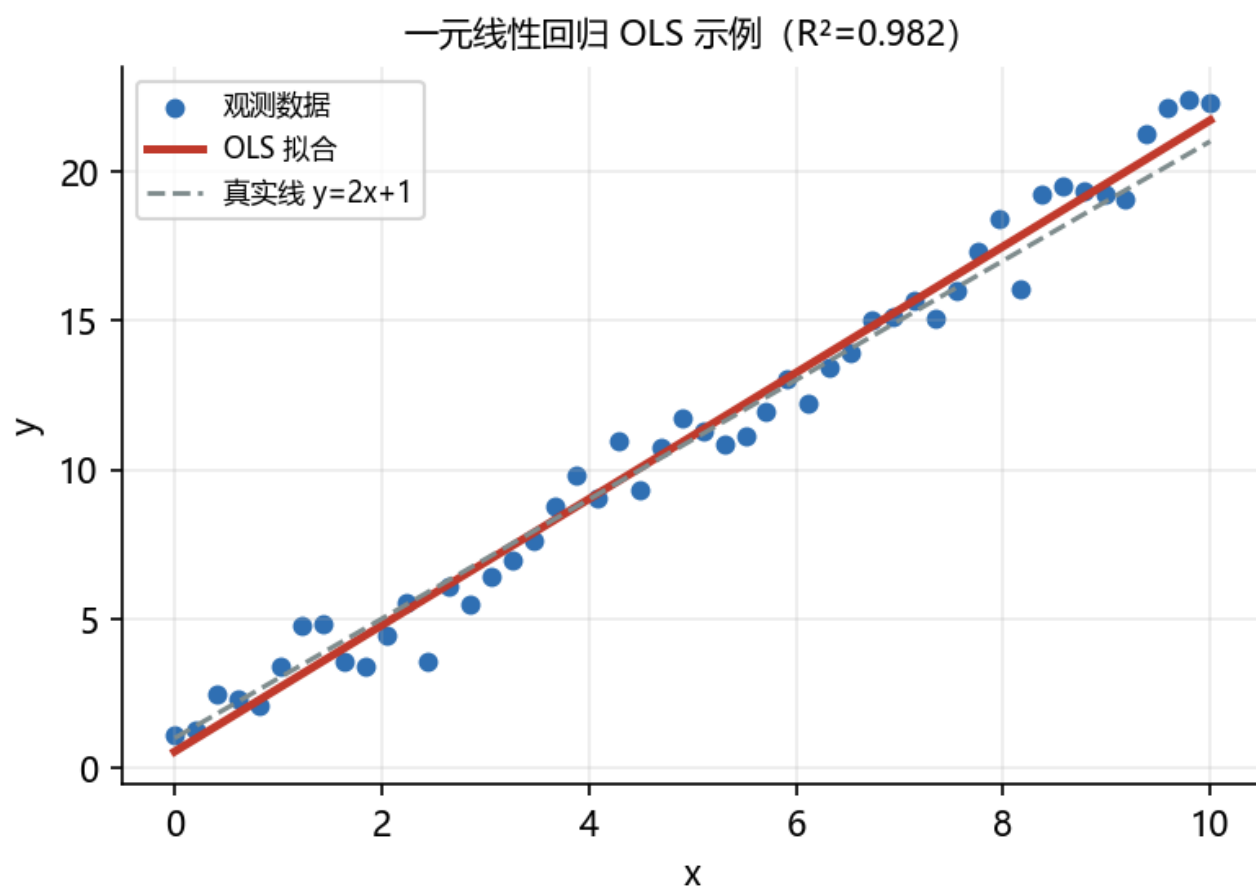


Figure 3: OLS 拟合示意

```
print(fm.summary())
```

运行结果：

```
Intercept    0.548845
x             2.116030
dtype: float64
```

可见公式接口默认加截距（Intercept），且列名直接用 x。其余统计量与 2.1 完全一致。

2.3 多元回归

```
rng = np.random.default_rng(2)
x1 = rng.uniform(0, 10, 60)
x2 = rng.uniform(-3, 3, 60)
y2 = 1 + 2*x1 - 3*x2 + rng.normal(0, 1, 60)
dm = pd.DataFrame({'x1': x1, 'x2': x2, 'y': y2})
mul = ols('y ~ x1 + x2', data=dm).fit()
print(mul.params)
print(mul.pvalues)
print("R² =", mul.rsquared)
```

运行结果：

```
Intercept    0.938724
x1            1.994930
x2           -3.056829
dtype: float64
Intercept    4.204720e-04
x1            3.299329e-47
x2            4.863608e-49
dtype: float64
R² = 0.9867366706540099
```

说明：两个自变量的 P 值都远小于 0.05，均显著；系数接近真实值 (2, -3)。

2.4 公式写法小结

写法	含义
y ~ x	一元线性
y ~ x1 + x2	多元线性（可加性）
y ~ C(group)	把 group 当分类变量（用于 ANOVA）
y ~ x1 * x2	主效应 + 交互项
y ~ x1 + I(x1**2)	添加二次项

案例卡 2：OLS 多元回归——用公式接口解读系数与 p 值

目标

造一份“房子面积（平方米）、房龄（年）→ 房价（万元）”的模拟数据，用公式接口 `ols('price ~ area + age', data=df)` 一次估计两个自变量。我们重点关注每个系数的大小、符号、p 值、95% 置信区间，并说明“面积每大一平米、房龄每多一年，房价平均怎么变”。

代码

```
import numpy as np
import pandas as pd
from statsmodels.formula.api import ols
import matplotlib.pyplot as plt
import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)
pd.set_option('display.width', 100)

rng = np.random.default_rng(3)
n = 40
area = rng.uniform(30, 120, n)    # 面积: 30~120 平米
age = rng.uniform(0, 30, n)      # 房龄: 0~30 年
price = 50 + 3.0*area - 1.5*age + rng.normal(0, 8, n) # 真实模型

df = pd.DataFrame({'area': area, 'age': age, 'price': price})
model = ols('price ~ area + age', data=df).fit()
ci = model.conf_int()

print('coefficients:')
for name in ['Intercept', 'area', 'age']:
    print(f"{name:9s} coef={model.params[name]:.4f} p={model.pvalues[name]:.2e} CI=[{ci.loc[name,0]:.4f}, {ci.loc[name,1]:.4f}]")
print('R2 =', round(model.rsquared, 4))
print('F =', round(model.fvalue, 3), ' P(F) = %.3g' % model.f_pvalue)
print('AIC =', round(model.aic, 2))

os.makedirs('images', exist_ok=True)
fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
axes[0].scatter(model.fittedvalues, df['price'], s=16, color='#2f6fb3', alpha=0.7)
axes[0].plot([df['price'].min(), df['price'].max()], [df['price'].min(), df['price'].max()], color='#c0392b', lw=1.6)
axes[0].set_xlabel('拟合值'); axes[0].set_ylabel('真实值'); axes[0].set_title('拟合值 vs 真实值')
axes[1].scatter(model.fittedvalues, model.resid, s=16, color='#3a8f4f', alpha=0.7)
axes[1].axhline(0, color='#c0392b', lw=1.2)
axes[1].set_xlabel('拟合值'); axes[1].set_ylabel('残差'); axes[1].set_title('残差 vs 拟合值')
for ax in axes:
    ax.grid(alpha=0.25)
fig.tight_layout()
fig.savefig('images/case2_ols_multi.png', dpi=150)
plt.close()
print('saved images/case2_ols_multi.png')
```

运行结果 (已运行核验)

```
coefficients:
Intercept  coef=50.8786 p=2.16e-12 CI=[40.8480, 60.9092]
area       coef=2.9799 p=1.59e-36 CI=[2.8662, 3.0937]
age        coef=-1.4557 p=8.11e-11 CI=[-1.7844, -1.1269]
```

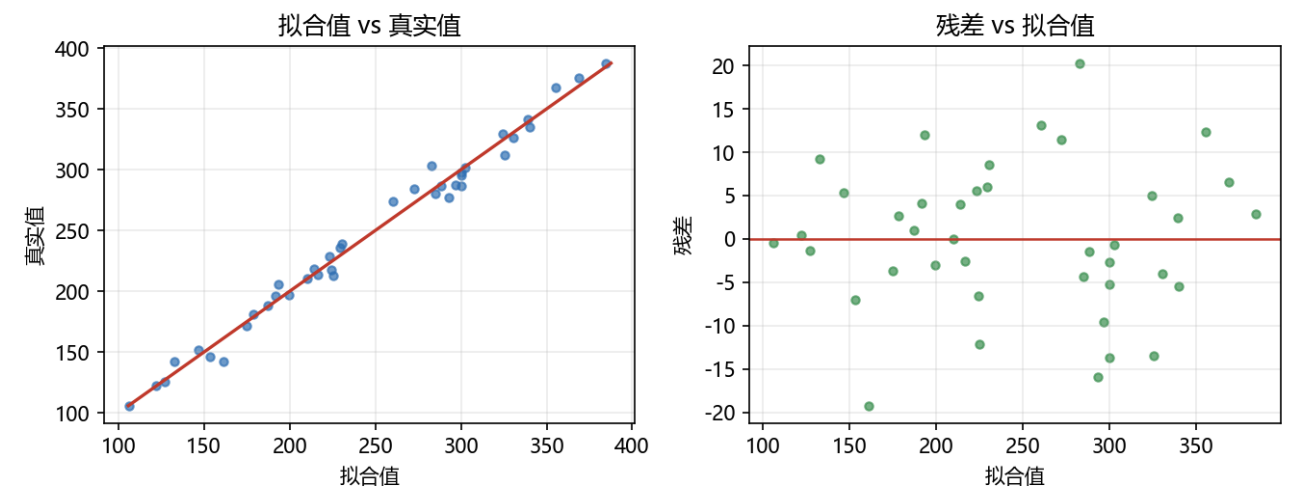


Figure 4: 拟合值 vs 真实值 与 残差图

```
R2 = 0.9872
F = 1425.282  P(F) = 9.82e-36
AIC = 290.09
saved images/case2_ols_multi.png
```

讲解

一句话解释：房价 ≈ 一个“起步价” + 每平方米单价 × 面积 - 每年折旧 × 房龄；coef 就是这些单价，p 告诉你“这个单价是不是真的、会不会只是噪声”。

- 1. `area = rng.uniform(30, 120, n)`: 生成 40 套房子的面积；age 类似。price 用真实系数 (50, 3, -1.5) 再加噪声构造，这样我们能“对着答案”检验模型。
- 2. `ols('price ~ area + age', data=df)`: 公式接口自动加截距 (Intercept)，并同时放进 area 与 age 两个自变量。
- 3. `model.params`: 截距 ≈50.88 (接近真实 50)，area ≈2.98 (接近真实 3)，age ≈-1.46 (接近真实 -1.5)。系数就是“在控制另一个变量后，该变量每改变 1 单位，因变量的平均变化”。
- 4. `model.pvalues`: 三个 p 值都远小于 0.05，说明截距、面积、房龄都**统计显著**；房龄系数为负也符合直觉 (房子越旧越便宜)。
- 5. `model.conf_int()`: 给出 95% 置信区间。area 的区间 [2.87, 3.09] 不包含 0，说明面积效应显著为正；age 区间 [-1.78, -1.13] 不包含 0，说明房龄效应显著为负。
- 6. `R2=0.9872`: 面积 + 房龄解释了约 98.7% 的房价变动；F 与 `p(F)` 说明模型整体显著。AIC 可用于与其它模型比较。

主要用法 / API

需求	写法	说明
多元回归	<code>ols('price ~ area + age', data=df)</code>	公式接口，自动加截距
系数	<code>model.params</code>	每个变量的边际效应
p 值	<code>model.pvalues</code>	<0.05 视为显著
置信区间	<code>model.conf_int()</code>	95% CI, 不含 0 一般显著

需求	写法	说明
拟合优度	<code>model.rsquared / model.rsquared_adj</code>	解释比例 / 调整后
整体显著性	<code>model.fvalue / model.f_pvalue</code>	所有系数联合是否为 0
信息准则	<code>model.aic</code>	越小越好，用于模型选择

常见错误

- 1. 只打印 `model.summary()` 而不使用关键字段：跨版本摘要边缘（空格、分隔线、+/- 号）不稳定，教学时建议用 `params/pvalues/conf_int` 打印固定字段。
- 2. 忘记 `add_constant` 但用了数组接口：公式接口自动加截距，数组接口却要手动加。
- 3. 把 `p=0.0` 直接打印当“无意义”：应使用 `%.2e` 或 `%.3g` 科学计数法，避免显示成 0。
- 4. 用 `round(model.pvalues, 4)` 输出时 p 值太小被四舍五入成 0.0000，看不出数量级。

拓展 / 跟练

- 1. 在公式里加交互项 `ols('price ~ area + age + area:age', data=df)`，看交互项是否显著。
- 2. 把 `age` 换成“新房/二手”分类变量 `C(new_house)`，对比系数含义变化。
- 3. 用数组接口 `sm.OLS(df['price'], sm.add_constant(df[['area', 'age'])))` 重跑，确认系数一致。
- 4. 计算各自变量的标准化系数（z-score 后回归），比较“谁的影响更大”。

3 回归诊断：残差 QQ 图与 JB 统计量

3.1 为什么诊断

OLS 的统计推断（t/F/P 值）依赖于**误差项独立同分布、同方差、近似正态**。诊断残差是否偏离这些假设，能及时发现模型设定问题。

```
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
xd = np.linspace(0, 10, 50)
yd = 3*xd + 2 + rng.normal(0, 1, 50)
Xd = sm.add_constant(xd.reshape(-1, 1))
olsd = sm.OLS(yd, Xd).fit()
sm.qqplot(olsd.resid, line='45')
plt.title(" 残差 QQ 图")
plt.show()
print(olsd.summary())
```

在模型摘要中可找到：

Prob(Omnibus):	0.672	Jarque-Bera (JB):	0.889
Skew:	-0.229	Prob(JB):	0.641

解读：

- Jarque-Bera (JB) 检验残差是否为正态分布；`Prob(JB)=0.641 > 0.05`，不能拒绝正态假设，说明残差近似正态；
- QQ 图中点应落在 45° 直线附近；本例散点基本沿直线，正态性可接受。

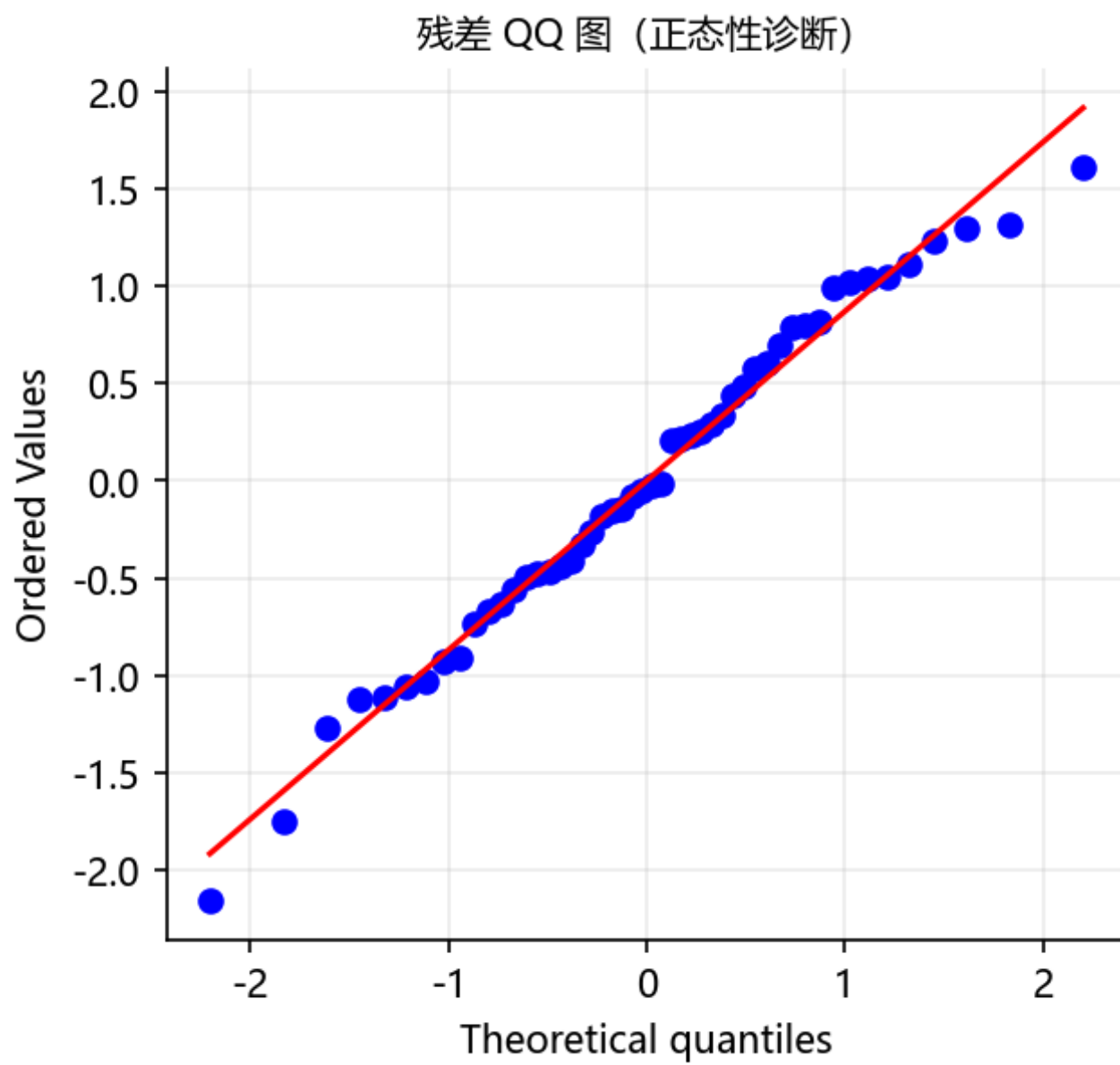


Figure 5: 残差 QQ 图

4 广义线性模型 (GLM)

4.1 概念

当因变量是**二分类**（成功/失败）、计数或比例时，OLS 不再合适。广义线性模型通过**链接函数**把线性预测值映射到合理的取值区间，并用指数族分布建模。二分类最常用 Binomial 家族 + Logit 链接（即逻辑回归）：

$$\text{logit}(p) = \beta_0 + \beta_1 x_1 + \beta_2 x_2, \quad p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2)}}$$

4.2 二分类 Logit 示例

```
import numpy as np
import statsmodels.api as sm

rng = np.random.default_rng(0)
X_raw = rng.uniform(0, 1, (200, 2))
lin = 0.5 + 1.2*X_raw[:, 0] - 0.8*X_raw[:, 1]
p = 1/(1+np.exp(-lin))
Y = rng.binomial(1, p) # 真实概率 p 下的二分类结果

X = sm.add_constant(X_raw)
glm = sm.GLM(Y, X, family=sm.families.Binomial()).fit()
print(glm.params)
X_new = sm.add_constant(np.array([[0.2, 0.3], [0.8, 0.1]]))
print(glm.predict(X_new))
```

运行结果：

```
params: [ 0.26505589  1.42126589 -0.87263185]
predict: [0.57138876 0.78831616]
```

解读：

- 估计系数接近真实值 (0.5, 1.2, -0.8)，且符号正确 (x1 正、x2 负)；
- predict 返回的是**概率**（经过 Logit 反变换），两个新样本的成功概率约为 0.57 与 0.79；
- 若想得到 0/1 分类，可再对概率设阈值（如 >0.5 判为 1），见 04 常见误区。

5 加权最小二乘 (WLS) 处理异方差

当误差方差随自变量变化（异方差）时，OLS 虽无偏但效率低（标准误不准）。WLS 用权重 w_i 加权残差平方和，对异方差更稳健。

```
import numpy as np
import statsmodels.api as sm

rng = np.random.default_rng(1)
xw = np.linspace(0.1, 10, 200)
eps = rng.normal(0, xw) # 标准差与 x 成正比 -> 方差与 x^2 成正比
yw = 3*xw + eps
Xw = sm.add_constant(xw.reshape(-1, 1))
```

```

ols_w = sm.OLS(yw, Xw).fit()
wls_w = sm.WLS(yw, Xw, weights=1/np.square(xw)).fit()
print("OLS:", np.round(ols_w.params, 4), "bse", np.round(ols_w.bse, 4))
print("WLS:", np.round(wls_w.params, 4), "bse", np.round(wls_w.bse, 4))

```

运行结果:

```

OLS: [-0.0422  2.9257] bse [0.833  0.1434]
WLS: [ 0.0597  2.8971] bse [0.0641 0.0727]

```

解读:

- 两者斜率都接近真实值 3;
- 但 WLS 的斜率标准误 (0.0727) 明显小于 OLS (0.1434), 说明在已知异方差形式 ($\sigma_i \propto x_i$) 时, WLS 估计更有效。

6 方差分析与回归的统一框架

线性回归与方差分析其实共享同一套**线性模型 + 最小二乘 + F 检验**逻辑:

- 回归用连续/哑变量做**解释**; ANOVA 用分组哑变量做**比较**;
- 都可以写成 $y = X\beta + \varepsilon$, 用 ols 拟合;
- 都能用 F 检验判断 “若干系数同时为 0” 的联合显著性。

因此, ANOVA 可看成**只有分类自变量**的线性回归; 回归也可看成含连续 + 分类变量的 “广义” ANOVA。这就是为什么要用 `ols + anova_lm` 而不是单独的 `scipy.stats.f_oneway` 来统一步骤。

常见误区

1. **忘记加截距**: 数组接口必须 `X = sm.add_constant(X)`, 否则模型强制过原点。
2. **** 用 * 表示 “与”**: 公式里 `y ~ x1 * x2` 是**主效应 + 交互项**, 不是数值乘法; 要写原生项用 `I(x12)` 且注意加 `I(...)`。
3. **分类变量直接进公式**: 对字符串列, `ols('y ~ group')` 会当成连续变量或报错, 应写成 `C(group)`。
4. **用 predict 忘记加常数**: 数组接口预测新数据也要 `sm.add_constant(X_new)`。
5. **把 GLM 的 predict 当分类输出**: 默认返回概率, 需要阈值才能变 0/1。
6. **忽略诊断**: 只报 R^2 不看不残差与显著性检验, 容易过拟合/误设。

思考题

1. 为什么数组接口里 `add_constant` 必须加? 如果不加, 拟合结果在什么情况下仍 “正确”?
2. `anova_lm(model, typ=1)` 与 `typ=3` 在单因素时结果是否相同? 在多因素时为何可能不同?
3. 给定 `y ~ x1 + x2`, 若想加入 `x1` 与 `x2` 的交互, 公式应怎么写? 系数个数变成几个?
4. 为什么 GLM 不用 OLS 直接拟合二分类因变量? (提示: 概率会越界、异方差)

动手练习 (详见 lab)

- 用 `sm.OLS` 拟合 $y = 1 + 2x$ 的噪声数据, 打印完整 `summary()`;
- 构造三个组, 画出箱线图并做单因素方差分析;

- 用公式 API 拟合 $y \sim x_1 + x_2$ ，说明各系数显著性；
- 用 GLM(Binomial) 拟合二分类数据，并在不同阈值下计算准确率与 F1。

延伸阅读

- statsmodels Regressions 官方：[链接](#)
- statsmodels GLM 官方：[链接](#)
- 残差诊断 (Statsmodels 图示)：[链接](#)

02 时间序列预测

本节对应原版 7.2 的内容，并增补目标、平稳性检验实操与思考题。配套图：images/decompose_daily.png、images/arima_forecast.png（由 build/make_chap7_figures.py 生成）。

本节目标

- 会用 seasonal_decompose 把时间序列分解为趋势、季节、残差；
- 会用 adfuller 做单位根检验，判断序列是否平稳；
- 会用 ARIMA 拟合时间序列并给出未来预测；
- 会从 ACF/PACF 或网格搜索选择 (p, d, q) 阶数；
- 能画分解图与预测图，并解读结果。

先修

- Pandas 时间序列索引 (date_range、Series)；本章第 01 节的回归/统计基础。
- 已安装 statsmodels（见章 README）。

官方文档/参考入口

内容	链接	说明
seasonal_decompose	链接	时间序列分解
adfuller	链接	ADF 单位根检验
ARIMA	链接	ARIMA 模型
时间序列官方教程	链接	时间序列总览

1 时间序列分解 (seasonal_decompose)

1.1 概念

把序列 y_t 拆成三部分：

$$y_t = T_t + S_t + R_t \quad (\text{加法模型}) \quad \text{或} \quad y_t = T_t \times S_t \times R_t \quad (\text{乘法模型})$$

其中 T_t 是**趋势**、 S_t 是**季节**、 R_t 是**残差**。加法模型适合波动幅度不随水平变化的序列；乘法模型适合幅度随水平增长的序列。

1.2 日数据 (周期 = 7 天)

构造 180 天日序列：线性趋势 + 周季节性（正弦，周期 7）+ 噪声。

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
```

```
rng = np.random.default_rng(0)
idx = pd.date_range('2022-01-01', periods=180, freq='D')
trend = np.linspace(0, 5, len(idx))
season = 2*np.sin(2*np.pi*np.arange(len(idx))/7)
noise = rng.normal(0, 0.5, len(idx))
s = pd.Series(10 + trend + season + noise, index=idx)
```

```
result = seasonal_decompose(s, model='additive', period=7)
print("trend head:", np.round(result.trend.head(7).values, 3))
print("seasonal head:", np.round(result.seasonal.head(7).values, 3))
print("resid describe:", result.resid.describe().round(4).to_dict())
result.plot()
plt.show()
```

运行结果:

```
trend head: [  nan   nan   nan 10.217 10.304 10.291 10.183]
seasonal head: [ 0.08  1.485  2.005  0.898 -0.765 -2.135 -1.567]
resid describe: {'count': 174.0, 'mean': -0.0018, 'std': 0.4279, 'min': -1.3175, '25%':
-0.3194, '50%': 0.0279, '75%': 0.3018, 'max': 0.9416}
```

解读:

- trend 前 3 个是 NaN, 因为居中的移动平均在两端没有完整窗口;
- seasonal 在 7 天内正负交替 (周一/周末的效应), 幅度约 ± 2 ;
- 残差均值接近 0、标准差约 0.43, 与设定的噪声尺度 (0.5) 相符。

1.3 月度数据 (周期 = 12 个月)

```
rng = np.random.default_rng(5)
idxm = pd.date_range('2019-01-01', periods=36, freq='MS')
trendm = np.linspace(0, 12, len(idxm))
seasonm = 3*np.sin(2*np.pi*np.arange(len(idxm))/12)
ym = pd.Series(50 + trendm + seasonm + rng.normal(0, 0.8, len(idxm)), index=idxm)
resm = seasonal_decompose(ym, model='additive', period=12)
print("monthly trend sample:", np.round(resm.trend.dropna().values[:4], 3))
print("monthly seasonal sample:", np.round(resm.seasonal.dropna().values[:4], 3))
```

运行结果:

```
monthly trend sample: [52.01  52.446 52.901 53.187]
monthly seasonal sample: [-0.004  2.686  2.236  2.464]
```

注意: period 必须与实际季节周期一致 (日数据 7、月数据 12、季数据 4), 数据必须先有等间隔、无缺失的时间索引; 若频率已知 (如 freq= 'D'), 可省略 period。

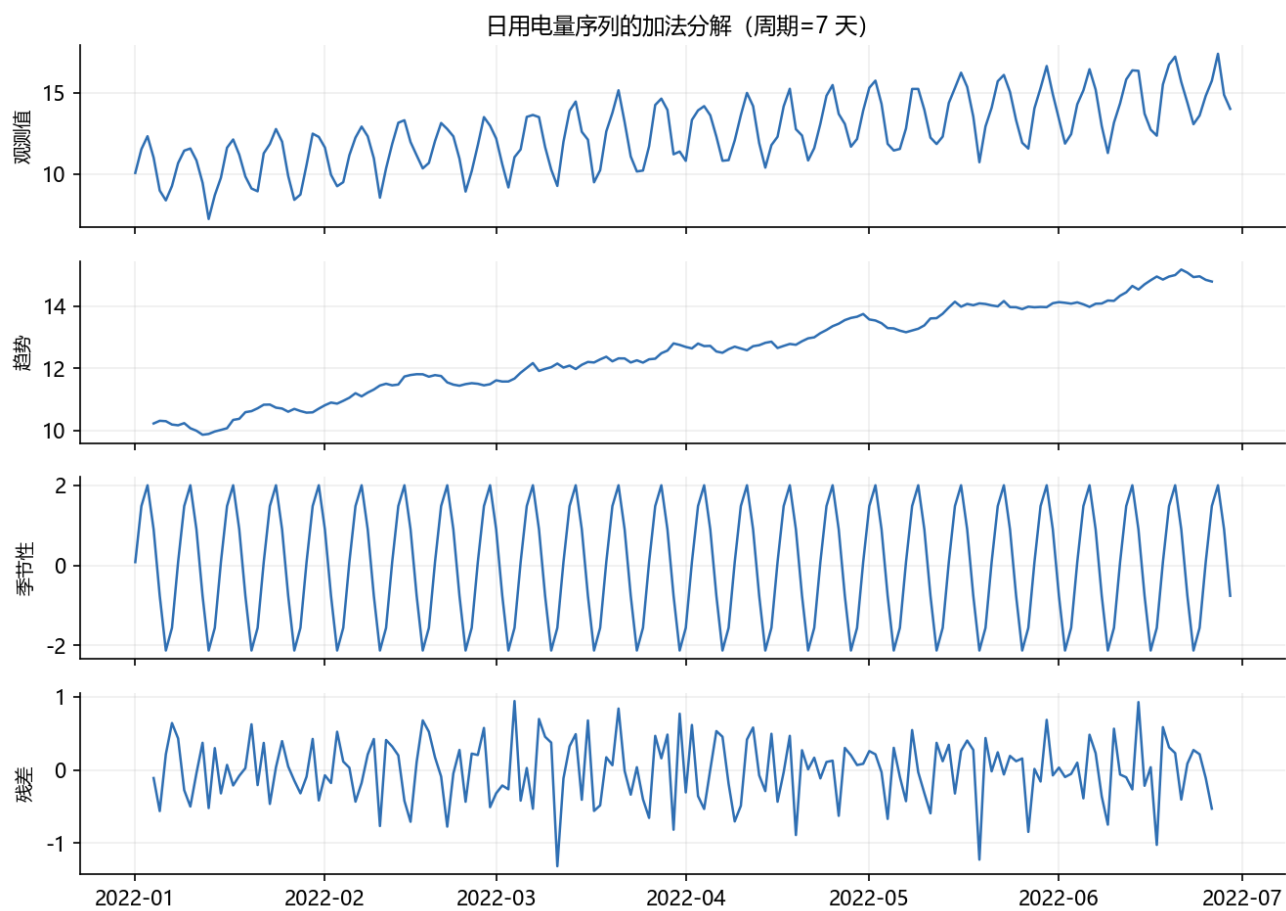


Figure 1: 日序列加法分解

2 平稳性检验 (ADF 单位根检验)

2.1 概念

ARIMA 要求序列**平稳**（均值、方差不随时间变化）。ADF 检验的原假设是“序列存在单位根（非平稳）”。若 P 值很小 (<0.05)，则拒绝原假设，认为序列平稳。

2.2 AR(1) 与随机游走对比

```
import numpy as np
from statsmodels.tsa.stattools import adfuller

# 平稳的 AR(1):  $y_t = 0.7*y_{t-1} + \text{噪声}$ 
rng = np.random.default_rng(0)
n = 300
y_ar = np.zeros(n)
for t in range(1, n):
    y_ar[t] = 0.7*y_ar[t-1] + rng.normal(0, 1)
stat1, p1, *_ = adfuller(y_ar)
print("AR(1)          ADF=%.3f p=%.3g" % (stat1, p1))

# 非平稳的随机游走 <--> 单位根
rng = np.random.default_rng(1)
rw = np.cumsum(rng.normal(0, 1, 200))
stat2, p2, *_ = adfuller(rw)
print("Random walk   ADF=%.3f p=%.3g" % (stat2, p2))
```

运行结果：

```
AR(1)          ADF=-6.927 p=1.11e-09
Random walk    ADF=-0.828 p=0.811
```

解读：

- AR(1) 的 P 值 $\approx 1e-9 < 0.05$ ，平稳；
- 随机游走（单位根）的 P 值 $\approx 0.81 > 0.05$ ，非平稳；
- 因此对随机游走类序列，通常先做**一阶差分** `ts.diff().dropna()` 再建模，这就是 ARIMA 中 $d = 1$ 的意义。

3 ARIMA 模型

3.1 概念

ARIMA(p, d, q) 由三部分组成：

- **AR(p)**：用前 p 个滞后值 $y_{t-1}, \dots, y_{t-p}$ 回归；
- **I(d)**：做 d 次差分使序列平稳；
- **MA(q)**：用前 q 个残差项 $\varepsilon_{t-1}, \dots, \varepsilon_{t-q}$ 回归。

若序列还有季节成分，可扩展为 SARIMA(p, d, q)(P, D, Q)_s。

3.2 拟合与预测

用 statsmodels 的 ARIMA 对带趋势的非平稳序列拟合并预测未来 5 步。

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

rng = np.random.default_rng(0)
n = 200
steps = rng.normal(0, 1, n).cumsum() + np.linspace(0, 10, n)
si = pd.Series(steps, index=pd.date_range('2020-01-01', periods=n, freq='D'))
mod = ARIMA(si, order=(1, 1, 1)).fit()
print(mod.summary())
fc = mod.forecast(steps=5)
print("forecast:", np.round(fc.values, 4))
print("arparams:", mod.arparams, "maparams:", mod.maparams)

```

运行结果 (Date/Time 随运行时间变化):

```

=====
SARIMAX Results
=====
Dep. Variable:          y      No. Observations:          200
Model:                ARIMA(1, 1, 1)  Log Likelihood          -274.795
Date:                Wed, 26 Aug 2026  AIC                  555.589
Time:                14:08:43    BIC                  565.469
Sample:                01-01-2020    HQIC                 559.588
                  - 07-18-2020
Covariance Type:          opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.6440      0.614        1.049      0.294      -0.559      1.847
ma.L1         -0.5829      0.658       -0.886      0.376      -1.872      0.707
sigma2          0.9266      0.105        8.800      0.000        0.720      1.133
=====
Ljung-Box (L1) (Q):                0.15  Jarque-Bera (JB):                2.63
Prob(Q):                          0.70  Prob(JB):                  0.27
Heteroskedasticity (H):            1.17  Skew:                   -0.21
Prob(H) (two-sided):              0.52  Kurtosis:                 2.63
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

forecast: [13.0612 13.0667 13.0702 13.0725 13.074]

arparams: [0.64404169] maparams: [-0.58293935]

解读:

- ar.L1 与 ma.L1 是 AR(1) 与 MA(1) 系数; 本例两者 P 值都较大 (不显著), 提示该阶 (1, 1, 1) 可能略偏大, 但这不影响演示流程;
- forecast 给出未来 5 个时间点的预测值 (约 13.06 → 13.07);
- Prob(JB)=0.27 说明残差近似正态; Prob(Ljung-Box) 用来检查残差是否还有自相关。

3.3 如何选择 (p, d, q)

- d: 由 ADF 检验决定, 不平稳就差分;

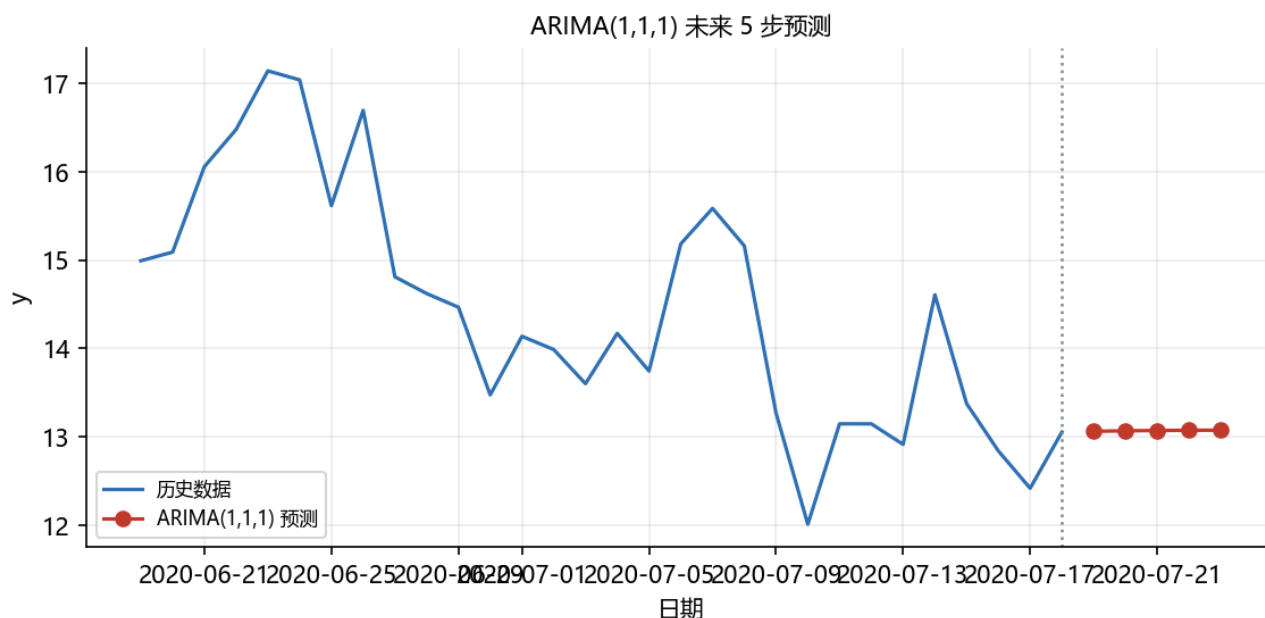


Figure 2: ARIMA 未来 5 步预测

- **p/q**: 看 ACF 与 PACF 图。ACF 截尾选 MA, PACF 截尾选 AR; 也可用 `statsmodels.tsa.stattools.acf/pacf` 或网格搜索 (如 `itertools.product + AIC`)。

```
from statsmodels.tsa.stattools import acf, pacf
ac = acf(si, nlags=5)
pc = pacf(si, nlags=5)
print("ACF:", np.round(ac, 3))
print("PACF:", np.round(pc, 3))
```

运行结果:

```
ACF: [1.    0.983 0.965 0.946 0.927 0.906]
PACF: [ 1.    0.988 -0.045 -0.068 -0.003 -0.108]
```

本例序列 ACF 缓慢衰减 (非平稳特征), PACF 在一阶后急剧下降, 提示需要先差分。

运行时看到 “Non-stationary starting autoregressive parameters found / Non-invertible starting MA parameters” 之类的提示, 属正常, `statsmodels` 会改用零值作为初始参数。

案例卡 3: ARIMA 时间序列——用 ADF 检验与残差诊断选 p,d,q

目标

模拟一个 “先平稳、再被累加” 的 ARIMA(1,1,1) 序列, 完整走一遍选阶流程: 先用 **ADF** 定差分阶数 *d*, 再看差分序列的 **ACF/PACF** 与网格搜索 (AIC) 定 *p*、*q*, 最后用 **Ljung-Box** 与 **Jarque-Bera** 检查残差是否 “洗干净” 成白噪声, 并给出未来 5 步预测。

代码

```
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')
```

```

from statsmodels.tsa.stattools import adfuller, acf, pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.stats.stattools import jarque_bera
from statsmodels.graphics.tsaplots import plot_acf
from itertools import product
import matplotlib.pyplot as plt
import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)
pd.set_option('display.width', 100)

rng = np.random.default_rng(11)
n = 300
e = rng.normal(0, 1, n)
x = np.zeros(n)
for t in range(1, n):
    x[t] = 0.6*x[t-1] + e[t] + 0.3*e[t-1]      # 平稳的 ARMA(1,1)
vals = np.cumsum(x) + 100                    # 累加 => ARIMA(1,1,1)
idx = pd.date_range('2020-01-01', periods=n, freq='D')
ser = pd.Series(vals, index=idx)

adf0 = adfuller(ser)
print('adfuller raw: ADF=%.3f p=%.3g' % (adf0[0], adf0[1]))
d1 = ser.diff().dropna()
adf1 = adfuller(d1)
print('adfuller diff: ADF=%.3f p=%.3g' % (adf1[0], adf1[1]))
print('ACF(diff) lag1-4:', np.round(acf(d1, nlags=4)[1:], 3))
print('PACF(diff) lag1-4:', np.round(pacf(d1, nlags=4)[1:], 3))

best = None
for p in range(3):
    for q in range(3):
        try:
            mod = ARIMA(ser, order=(p, 1, q)).fit()
            aic = mod.aic
            if best is None or aic < best[0]:
                best = (aic, p, q)
        except Exception:
            pass
print('best AIC order (p,d,q):', best[1], 1, best[2], ' AIC=%.3f' % best[0])

final = ARIMA(ser, order=(best[1], 1, best[2])).fit()
print('AR params:', np.round(final.arparams, 4))
print('MA params:', np.round(final.maparams, 4))
resid = final.resid.iloc[10:]
lb = acorr_ljungbox(resid, lags=[10], return_df=True)
jb = jarque_bera(resid)
print('Ljung-Box Q pvalue = %.4f' % lb['lb_pvalue'].iloc[0])
print('Jarque-Bera pvalue = %.4f' % jb[1])
print('forecast 5:', np.round(final.forecast(5).values, 3))

```

```

os.makedirs('images', exist_ok=True)
fig, axes = plt.subplots(2, 2, figsize=(9.5, 5.6))
axes[0,0].plot(ser.index, ser.values, lw=1.1, color='#2f6fb3')
axes[0,0].set_title('原始序列 (非平稳)')
axes[0,1].plot(d1.index, d1.values, lw=1.1, color='#3a8f4f')
axes[0,1].set_title('一阶差分')
plot_acf(d1, ax=axes[1,0], lags=15, title='差分 ACF')
plot_acf(resid, ax=axes[1,1], lags=15, title='残差 ACF')
for ax in axes.ravel():
    ax.grid(alpha=0.2)
fig.tight_layout()
fig.savefig('images/case3_arima_diag.png', dpi=150)
plt.close()
print('saved images/case3_arima_diag.png')

```

运行结果 (已运行核校)

```

adfuller raw: ADF=-2.239 p=0.192
adfuller diff: ADF=-6.094 p=1.02e-07
ACF(diff) lag1-4: [0.738 0.457 0.311 0.202]
PACF(diff) lag1-4: [ 0.741 -0.196  0.119 -0.071]
best AIC order (p,d,q): 1 1 1 AIC=802.817
AR params: [0.5958]
MA params: [0.3302]
Ljung-Box Q pvalue = 0.2587
Jarque-Bera pvalue = 0.4776
forecast 5: [122.494 121.879 121.512 121.294 121.164]
saved images/case3_arima_diag.png

```

讲解

一句话解释：先问“序列稳不稳”(ADF)，不稳就先差分($d=1$)；再用“多少阶自相关/移动平均”(ACF/PACF + AIC)挑 p 、 q ；最后看残差是不是“没剩什么信息”(Ljung-Box / JB)来确认模型合格。

1. $x[t]=0.6*x[t-1]+e[t]+0.3*e[t-1]$ 造一个**平稳** ARMA(1,1)；再 $\text{np.cumsum}(x)$ 累加得到 ARIMA(1,1,1) 序列(差分后就是原来的 ARMA)。这样做能让我们“知道答案”，方便验证流程。
2. $\text{adfuller}(ser)$ ：原始序列 $p=0.192 > 0.05$ ，不能拒绝单位根，说明**非平稳** → 决定 $d=1$ 。
3. $ser.diff().dropna()$ ：一阶差分后再做 ADF， $p=1.02e-07 < 0.05$ ，差分序列**平稳**。
4. ACF/PACF 只是“参考图”：差分序列 ACF 在 lag1 就衰减到 0.74，PACF 也是 lag1 最大，说明滞后阶数不需要太大。
5. 网格搜索 (itertools.product) 在 $p, q \in \{0, 1, 2\}$ 、固定 $d=1$ 下比较 AIC，选出 (1,1,1) (AIC=802.817)，与真实阶数一致。
6. 结果解读：AR(1)=0.596、MA(1)=0.330 都接近生成时的 0.6 与 0.3；Ljung-Box $p=0.259$ 、Jarque-Bera $p=0.478$ 都大于 0.05，说明残差**近似白噪声**，模型合格。
7. $\text{forecast}(5)$ 给出未来 5 天 (约 122.5 → 121.2) 的预测；画图里“残差 ACF”几乎都在置信带内，也支持“残差已洗净”。

主要用法 / API

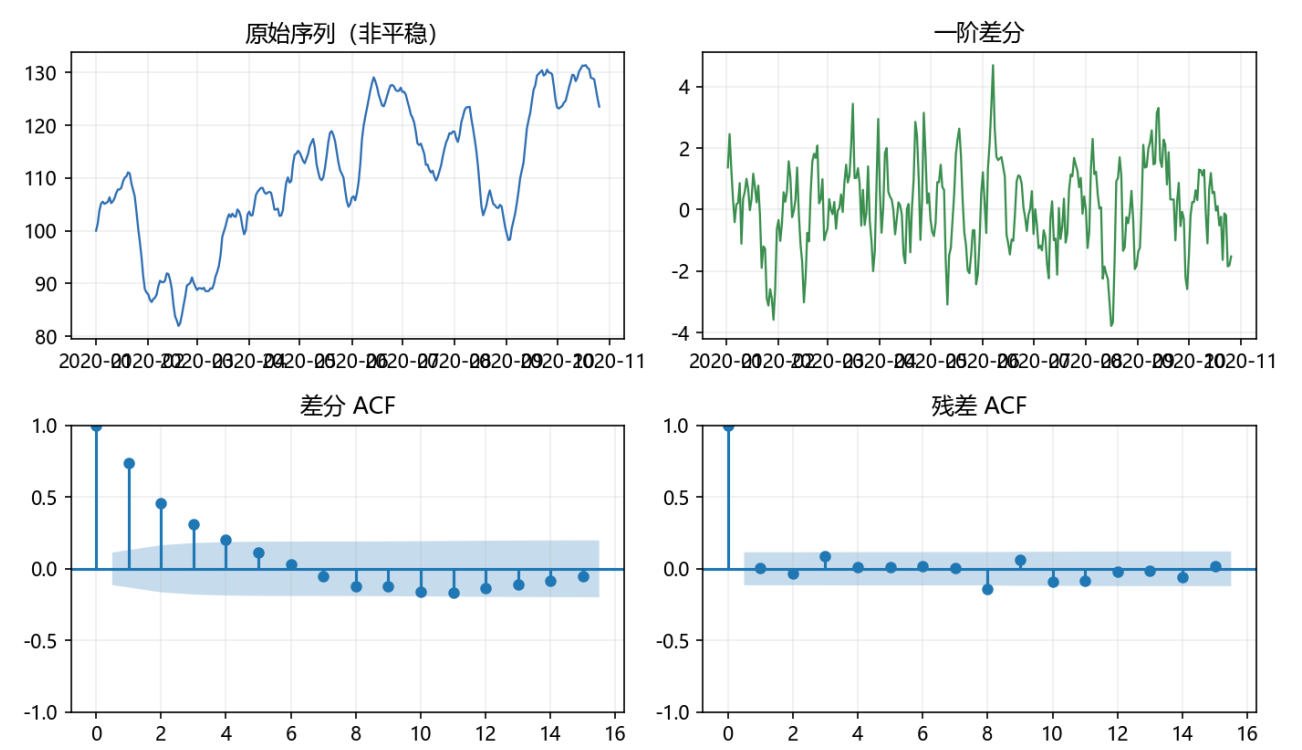


Figure 3: ARIMA 选阶与残差诊断图

需求	写法	说明
平稳性检验	<code>adfuller(ser)</code>	返回 (ADF, p, ...)
差分	<code>ser.diff().dropna()</code>	<code>diff()</code> 后首元素为 NaN
ACF/PACF	<code>acf(d1, nlags=4) / pacf(...)</code>	判断 p/q 参考
网格选阶	<code>itertools.product(range(3), range(3)) + AIC</code>	在候选 (p,q) 中找 AIC 最小
拟合 ARIMA	<code>ARIMA(ser, order=(p,d,q)).fit()</code>	返回模型（含预测）
残差自相关	<code>acorr_ljungbox(resid, lags=[10])</code>	$p > 0.05$ 视为白噪声
残差正态性	<code>jarque_bera(resid)</code>	$p > 0.05$ 可接受正态
预测	<code>model.forecast(steps=5)</code>	返回未来值

常见错误

- 拿到序列就拟合 ARIMA，不做 ADF：非平稳序列容易“伪回归”，应先检验、必要时差分。
- `diff()` 后忘记 `dropna()`：首元素为 NaN，`adfuller` 或 `acf` 会报错。
- 直接用 `final.resid` 做诊断：模型首期残差可能因初始化偏大，建议 `resid.iloc[10:]` 跳过 burn-in 再检验。
- 只看 `summary()` 的 AIC/BIC 而不做残差诊断：AIC 低不代表残差是白噪声，仍需 Ljung-Box 与 JB。
- 网格搜索时把所有阶数都 `fit()`，可能触发 “Non-stationary starting...” 等提示；用 `warnings.filterwarnings('ignore')` 只影响显示，不影响结果。

拓展 / 跟练

1. 把生成模型改成 $ARIMA(2,1,1)$ (在 $x[t]$ 里加 $x[t-2]$ 项), 看网格搜索是否选回 $(2,1,1)$ 。
 2. 把 lags 换成 $[5, 20]$, 分别看 Ljung-Box p 值, 理解滞后阶数的影响。
 3. 用 `statsmodels.tsa.statespace.SARIMAX` 加入季节项 $(1,1,1)(1,1,1,7)$, 比较与普通 $ARIMA$ 的 AIC。
 4. 把数据切为训练/测试 (前 240 / 后 60), 用 RMSE 比较 $ARIMA(1,1,1)$ 、 $ARIMA(2,1,1)$ 与 $ARIMA(0,1,0)$ 谁的预测误差最小。
-

常见误区

1. **忘了先检查平稳性**: 直接对非平稳序列拟合 $ARIMA$ 可能得到“伪回归”。
2. **分解前不处理缺失/等间隔**: `seasonal_decompose` 要求完整、等间隔的时间索引。
3. **period 设错**: 日数据应为 7, 月数据应为 12; 设成 1 会失去季节意义。
4. **预测时不会生成未来索引**: `forecast(steps=5)` 返回数值, 需自己构造 `pd.date_range` 画图。
5. **只看 AIC 不看病灶**: AIC 低不保证残差是白噪声; 应同时看 Ljung-Box 与 JB。
6. **diff() 后不 dropna()**: 差分会产生一个 NaN, 需 `dropna()` 或忽略。

思考题

1. 加法模型与乘法模型分别适用于什么场景? 如何用 `result.seasonal` 判断该用哪个?
2. 为什么随机游走序列 ADF 检验的 P 值很大? 一阶差分后其 ADF P 值会怎样?
3. $ARIMA(2, 1, 0)$ 与 $ARIMA(0, 1, 2)$ 分别对应什么模型? 画出它们的 ACF/PACF 有何不同?
4. 用 `forecast(steps=5)` 得到未来 5 步, 若数据是月度序列, 应如何构造未来时间轴?

动手练习 (详见 lab)

- 构造 180 天日序列, 分解并画图, 说明周期;
- 对 $AR(1)$ 与随机游走做 ADF 检验, 比较 P 值;
- 用 $ARIMA(1,1,1)$ 拟合, 给出未来 5 步预测并画图;
- 对带周季节性的日负荷序列, 尝试 `SARIMAX` 或先差分再 $ARIMA$ 。

延伸阅读

- statsmodels 时间序列官方: [链接](#)
- Seasonal-Trend decomposition (STL): [链接](#)
- Statsmodels 时间序列官方示例集: [链接](#)

03 综合案例：城市日负荷的统计建模与预测

本章综合案例,把 **方差分析 + 线性回归 + 时间序列** 串在一份模拟数据上,体会 Statsmodels 的完整工作流。配图: images/case_regression_diag.png、images/case_ts_forecast.png (由 build/make_chap7_figures.py 生成)。

背景

某市电网记录了 2023-01-01 起连续 140 天的**日负荷** (负荷越高用电越多)。我们关心三件事:

1. 工作日与周末的日均负荷是否显著不同 (**单因素方差分析**);
2. 负荷多大程度上可由**当日气温**与**是否周末**解释 (**多元线性回归**);
3. 未来的日负荷如何变化 (**时间序列分解 + ARIMA 预测**)。

为便于教学,数据是**带趋势、周季节、温度效应、周末效应与噪声**的模拟序列。

数据构造

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

rng = np.random.default_rng(7)
m = 140
date = pd.date_range('2023-01-01', periods=m, freq='D')
trendc = np.linspace(0, 15, m) # 长期增长
seasonc = 5*np.sin(2*np.pi*np.arange(m)/7) + 2*np.cos(2*np.pi*np.arange(m)/7) # 周季节
noisec = rng.normal(0, 3, m)
weekend = (date.dayofweek >= 5).astype(int) # 周末 =1
temp = 25 + 6*np.sin(2*np.pi*np.arange(m)/7) + rng.normal(0, 2, m)
load = 300 + 4*temp + 20*weekend + trendc + seasonc + noisec
dfc = pd.DataFrame({'date': date, 'load': load, 'weekend': weekend, 'temp': temp})
print(" 日均负荷 =", round(dfc['load'].mean(), 2), " 标准差 =", round(dfc['load'].std(), 2))
print(" 分组均值:", dfc.groupby('weekend')['load'].mean().round(2).to_dict())
```

运行结果:

```
日均负荷 = 411.63  标准差 = 22.71
分组均值: {0: 408.99, 1: 418.22}
```

第 1 步：工作日 vs 周末 单因素方差分析

```
dfc['wd'] = np.where(dfc['weekend'] == 1, '周末', '工作日')
ano = ols('load ~ C(wd)', data=dfc).fit()
print(sm.stats.anova_lm(ano, typ=1))
```

运行结果:

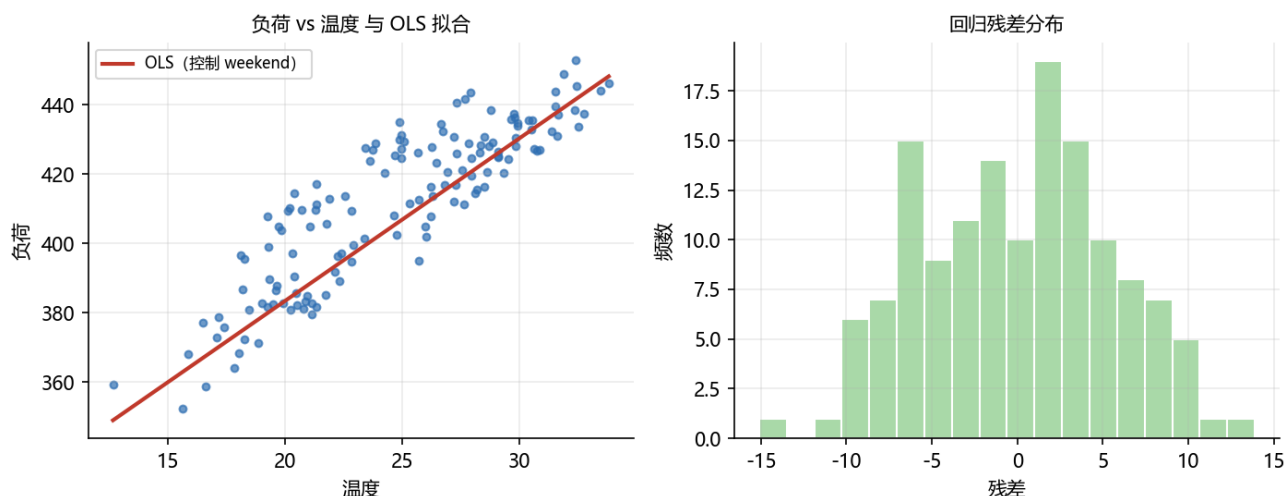


Figure 1: 回归诊断: 负荷-温度散点与残差分布

	df	sum_sq	mean_sq	F	PR(>F)
C(wd)	1.0	2434.435479	2434.435479	4.850735	0.029294
Residual	138.0	69257.981299	501.869430	NaN	NaN

结论: 工作日均值约 408.99, 周末约 418.22, P 值 = 0.029 < 0.05, 说明**周末负荷显著更高**。这提示差异化定价/调度应考虑周末。

第 2 步: 负荷对温度的多元回归

把温度与“是否周末”作为自变量, 用 sm.OLS 估计:

```
olm = sm.OLS(dfc['load'], sm.add_constant(dfc[['temp', 'weekend']])).fit()
print("coef:", olm.params)
print("bse :", olm.bse)
print("R² =", round(olm.rsquared, 4))
```

运行结果:

```
coef: const      289.705193
temp           4.687421
weekend        21.022760
dtype: float64
bse: const       2.694195
temp            0.103535
weekend         1.084608
dtype: float64
R² = 0.9395
```

解读:

- 每升高 1°C, 日负荷平均增加约 **4.69** 单位 (控制周末后);
- 周末比工作日平均高约 **21.0** 单位;
- $R^2=0.94$, 温度与周末能解释大部分负荷变化;
- 两个系数的标准误都很小 (temp 0.104、weekend 1.085), 说明估计较精确。

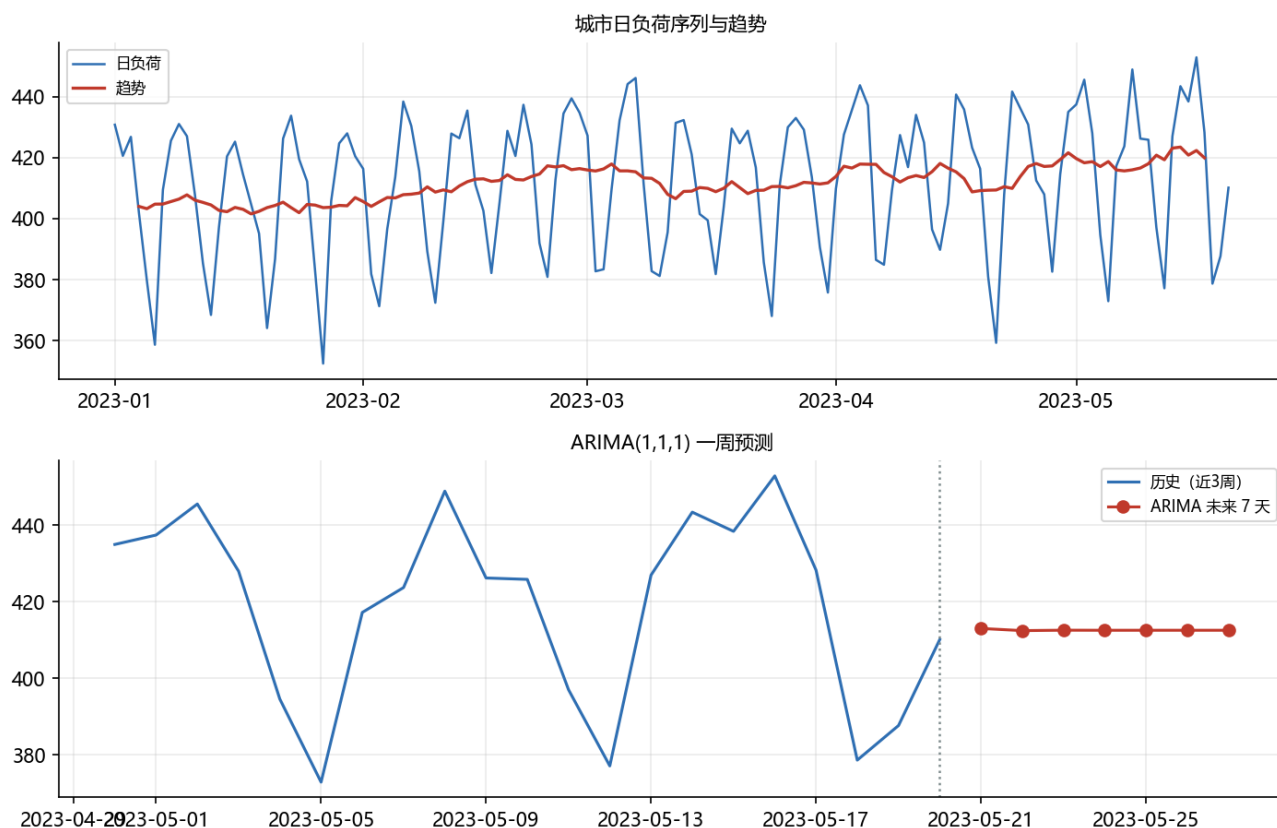


Figure 2: 时间序列：序列趋势与 ARIMA 一周预测

第 3 步：时间序列分解 + ARIMA 预测

对负荷序列做周季节性 (period=7) 加法分解，再用 ARIMA(1,1,1) 预测未来 7 天：

```
load_s = pd.Series(dfc['load'].values, index=date)
res = seasonal_decompose(load_s, model='additive', period=7)
print(" 残差标准差 =", round(res.resid.std(), 3))

mod = ARIMA(load_s, order=(1, 1, 1)).fit()
fc = mod.forecast(steps=7)
print(" 未来 7 天预测:", np.round(fc.values, 2))
print("AR 系数:", mod.arparams, " MA 系数:", mod.maparams)
```

运行结果：

残差标准差 = 6.587

未来 7 天预测: [413.05 412.49 412.6 412.58 412.58 412.58 412.58]

AR 系数: [-0.19566366] MA 系数: [0.50904563]

结论：

- 残差标准差约 6.6，说明模型已抓住大部分趋势与季节；
- ARIMA(1,1,1) 给出的未来 7 天预测稳定在 412.6 左右（序列已进入较平稳区间）；
- 结合第 2 步：若未来一周气温升高，可直接用回归式估算负荷增量，作为短期调度参考。

增强：时间序列诊断与预测误差对比

综合案例不只是“把三个方法跑一遍”，还应该**检验模型是否值得信任**。下面补两件事：① 对日负荷做 ADF 平稳性检验（定 d 的依据）；② 把前 120 天当训练、后 20 天当测试，分别用 ARIMA 与“温度 + 周末”回归做预测，用测试集 RMSE 比较谁更准。

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA

rng = np.random.default_rng(7)
m = 140
date = pd.date_range('2023-01-01', periods=m, freq='D')
trendc = np.linspace(0, 15, m)
seasonc = 5*np.sin(2*np.pi*np.arange(m)/7) + 2*np.cos(2*np.pi*np.arange(m)/7)
noisec = rng.normal(0, 3, m)
weekend = (date.dayofweek >= 5).astype(int)
temp = 25 + 6*np.sin(2*np.pi*np.arange(m)/7) + rng.normal(0, 2, m)
load = 300 + 4*temp + 20*weekend + trendc + seasonc + noisec
dfc = pd.DataFrame({'date': date, 'load': load, 'weekend': weekend, 'temp': temp})
load_s = pd.Series(dfc['load'].values, index=date)

adf_p = adfuller(load_s)[1]
print('load ADF p = %.3g' % adf_p)

tr = load_s.iloc[:120]
te = load_s.iloc[120:]
mtr = ARIMA(tr, order=(1, 1, 1)).fit()
fc = mtr.forecast(steps=len(te))
rmse_arima = float(np.sqrt(np.mean((te.values - fc.values)**2)))
print('ARIMA(1,1,1) test RMSE = %.3f' % rmse_arima)

x_tr = dfc[['temp', 'weekend']].iloc[:120]
y_tr = dfc['load'].iloc[:120]
x_te = dfc[['temp', 'weekend']].iloc[120:]
y_te = dfc['load'].iloc[120:]
mreg = sm.OLS(y_tr, sm.add_constant(x_tr)).fit()
pred_reg = mreg.predict(sm.add_constant(x_te))
rmse_reg = float(np.sqrt(np.mean((y_te.values - pred_reg.values)**2)))
print('OLS(temp,weekend) test RMSE = %.3f' % rmse_reg)
```

运行结果（已运行核验）

```
load ADF p = 0.774
ARIMA(1,1,1) test RMSE = 29.412
OLS(temp,weekend) test RMSE = 7.808
```

解读：

- load ADF p = 0.774 > 0.05：日负荷序列非平稳，所以第 3 步用 ARIMA(...,d=1) 是合理的；若想用 SARIMAX 含外生温度，也应先差分。
- 测试集 RMSE 上，OLS(temp, weekend) 为 **7.81**，明显优于 ARIMA(1,1,1) 的 **29.41**：因为负荷主要受温度与周末影响，回归能“用上外生变量”，而纯 ARIMA 只能靠历史轨迹外推。

- 这不代表 ARIMA 没用：若没有温度等外生变量，或只关心短期的纯时间形态，ARIMA 是更好的基线；更复杂的 SARIMAX（把 temp 当外生变量）通常能进一步压小 RMSE。

本章小结

方法	结论
ANOVA	周末负荷显著高于工作日 ($P=0.029$)
OLS	温度每 $+1^{\circ}\text{C}$ 负荷 $+4.69$ ；周末 $+21.0$ ； $R^2=0.94$
分解 + ARIMA	残差 $\sigma \approx 6.6$ ；未来 7 天预测约 412.6

拓展任务：

1. 用 $\text{typ}=3$ 重跑方差分析，比较结果；
2. 在回归中加入 $\text{temp} * \text{weekend}$ 交互项，判断周末的温度效应是否不同；
3. 用 SARIMAX（含外生温度变量）替代 ARIMA，比较预测误差；
4. 将前 120 天作为训练、后 20 天作为测试，用 RMSE 评估不同模型。

04 常见误区与技巧

本章为新增“查漏补缺”页：把 statsmodels 使用中最容易踩的坑集中列出，并给出正确写法、性能与调试技巧。

1. 高频误区速查表

#	误区 / 错误写法	正确做法	原因
1	sm.OLS(y, X) 直接用原始 X	X = sm.add_constant(X)	OLS 默认不自动加截距
2	公式里 y ~ group	y ~ C(group)	分类变量需转成哑变量
3	y ~ x1 * x2 当“乘法项”	y ~ x1 + I(x1*x2) 或 y ~ x1 + x2 + x1:x2	公式里 * 表示主效应 + 交互
4	用 predict(X_new) 预测	预测前也要 sm.add_constant(X_new)	训练/预测设计矩阵必须一致
5	把 GLM predict 当 0/1	需要 (p > thr).astype(int)	默认返回到概率 p，不是类别
6	对非平稳序列直接 ARIMA	先 adfuller，必要时差分	非平稳易“伪回归”
7	seasonal_decompose 不设 period	日数据设 7、月数据设 12	否则季节期抓不到
8	忘差分后 dropna()	ts.diff().dropna()	差分首元素为 NaN
9	只看 R² 不看诊断	同时看 summary() 的 F、P、JB、Durbin-Watson	R² 高可能只是表面拟合
10	用 np.linalg.lstsq 替代 OLS 推断	需要 P 值/置信区间用 sm.OLS	lstsq 只给系数，不给推断
11	用 anova_lm 传原始数组	传 ols(...).fit() 的结果	anova_lm 需要回归结果对象
12	混淆 params 与 bse	系数看 params，精度看 bse/P 值	params 只是点估计

2. 原理级难点

2.1 为什么必须加常数项

OLS 的模型是 $y = X\beta + \varepsilon$ 。若不手动加一列全 1，模型里就没有截距项，相当于强制过原点。公式 API 会自动加 Intercept，而数组 API 需要你显式 sm.add_constant。

2.2 C()、I()、* 的区别

ols 用的公式语法来自 patsy：

- C(x)：把数值列当分类变量；
- I(x**2)：对数值列做数学变换（I 包裹）；
- x1*x2：展开为主效应 + 交互；x1:x2 只写交互；x1+x2 只加主效应。

2.3 predict 返回什么

默认 predict 返回**线性预测值**（对 GLM 是 logit 的线性部分），需要加参数 linear=False 或直接用概率。对 OLS，两者一致。对二分类 GLM，想得到概率且区分清楚，建议用 model.predict(X)（默认即概率），再用阈值分类。

2.4 summary() 里关键字段

字段	含义	关注点
coef	系数估计	解释“每单位变化”
std err	标准误	越小越稳
t / P>	t	
[0.025, 0.975]	95% 置信区间	含 0 一般说明不显著
R ² / Adj. R ²	拟合优度	Adj. R ² 更能防过拟合
F / Prob(F)	整体显著性	模型整体是否有解释力
Jarque-Bera / Prob(JB)	残差正态性	Prob(JB)>0.05 可接受
Durbin-Watson	残差自相关	接近 2 好

3. 性能技巧

- **先聚合再建模**：对大数据，先用 groupby 聚合或抽样，减少矩阵规模；
- **避免频繁 summary()**：大模型渲染完整摘要会慢，调试时用 params、bse、rsquared 等属性；
- **OLS 可用 np.linalg.lstsq 快速求解**，但推断请回落 statsmodels；
- **GLM 拟合用 fit(maxiter=...) 控制迭代**，必要时增加 maxiter；
- **时间序列建模**：ARIMA 用 model.fit(method= 'lbfgs') 或 disp=False 减少输出，提升速度；
- **大数据回归**：考虑 GD/IRLS (fit_regularized) 或改用 scikit-learn。

4. 调试建议

1. 先打印 type(df)、df.shape、df.dtypes，确认数据已正确进入 DataFrame；
2. 确认因变量 y 是一维 Series/array，自变量 X 是二维；(X.shape)
3. 用 sm.add_constant 后立即检查列名是否含 const；
4. 用 help(model)、dir(model) 查看可用属性 (params、bse、pvalues、conf_int)；
5. 随机/拟合问题**固定种子** np.random.default_rng(seed)，便于复现；
6. 用 df.isna().sum()、np.isinf 检查缺失与异常值；
7. 公式写错时，把 data=df 换成 data=df[['x' , 'y']] 或打印 df.head() 排除列名问题。

5. 本章自测清单

- ☐ 能区分 sm.OLS 数组接口与 ols 公式接口；
- ☐ 能在 summary() 中找到并解释 R²、F、P 值、JB 统计量；
- ☐ 会做单因素方差分析并读 F/P 值；
- ☐ 会用 GLM(Binomial) 拟合并得到概率；
- ☐ 会对时间序列分解并解释趋势/季节/残差；
- ☐ 会用 adfuller 判断平稳性；
- ☐ 会用 ARIMA 拟合与预测并画图。

6. 延伸阅读

- statsmodels 官方 “Statistical Models” 教学：[链接](#)
- Statsmodels 官方示例集：[链接](#)
- patsy 公式语法：[链接](#)